



Experiment – 1.2.2

Student Name: Damandeep Singh

UID: 24BDA7O313

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 17-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To optimize state access and improve application performance using memoized selectors and efficient rendering strategies.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Framework/Libraries: React 18, Vite, Redux Toolkit (createSelector)
- Browser: Google Chrome (for testing and rendering)

3. Objective:

This experiment introduces concepts such as derived state, selector optimization, and minimizing unnecessary re-renders to ensure high performance in complex frontend applications.

4. Theory:

Every call to `useSelector` re-runs the selector function on every dispatched action. If a selector returns a newly created array or object each time (e.g. `state.posts.map(...)`), React-Redux sees a new reference and re-renders the component even if the underlying data is unchanged. Wrapping such derived selectors in `createSelector` caches the output by input reference, so the same array/object reference is returned — and the component does not re-render — until the actual input data changes.

5. Procedure/Algorithm:

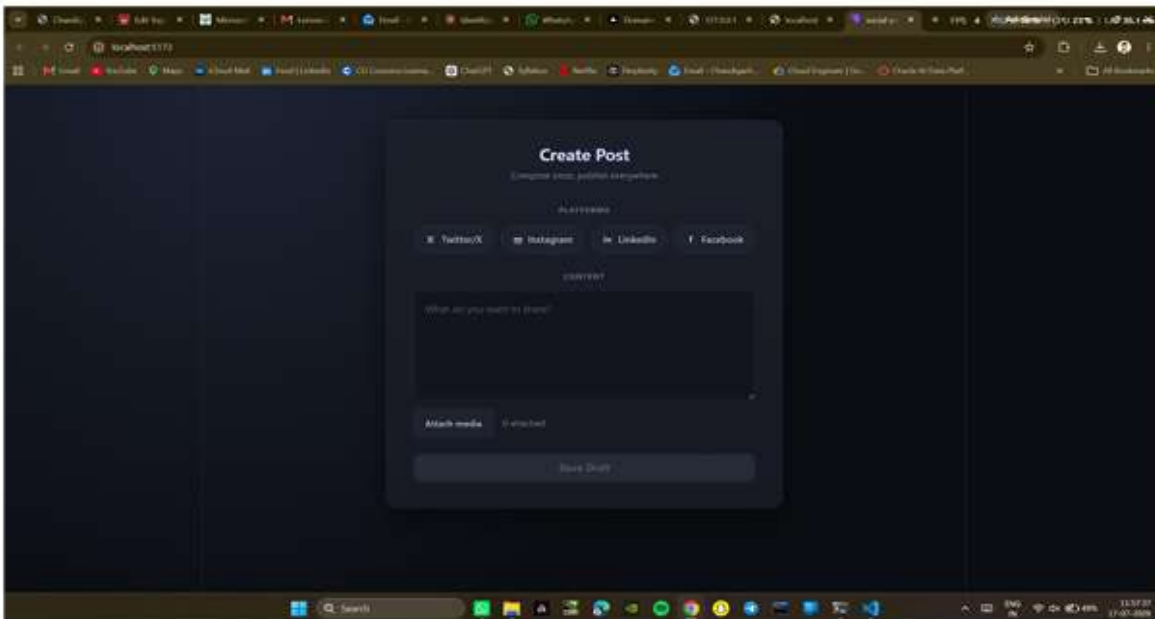


- Step 1: Identify all places in the UI where data is filtered, mapped, or aggregated from the store.
- Step 2: Replace inline component-level filtering with a dedicated selectors.js module.
- Step 3: Wrap each derived selector (selectAllPosts, selectAllDrafts, selectAllPlatforms) in createSelector.
- Step 4: Build compound selectors (selectPostCountByPlatform) from the base selectors rather than recomputing from raw state.
- Step 5: Verify in the UI that re-renders only occur when the relevant slice of state actually changes.

6. Code and Output:

```
export const selectAllDrafts = createSelector(
  [selectDraftsState],
  (drafts) => drafts.allIds.map((id) => drafts.byId[id])
);

export const selectPostCountByPlatform = createSelector(
  [selectAllPosts, selectAllPlatforms],
  (posts, platforms) => {
    const counts = Object.fromEntries(platforms.map((p) => [p.id, 0]));
    for (const post of posts) {
      for (const pid of post.platformIds) counts[pid] = (counts[pid] || 0) + 1;
    }
    return counts;
  }
);
```



7. Conclusion:

By routing every derived value through `createSelector`, the application avoids unnecessary recomputation and re-renders, keeping the UI responsive even as the number of drafts and posts grows — the Saved Drafts list and validation panels update smoothly with no visible lag during testing.

8. Learning Outcomes:

- Understood how referential equality affects React-Redux re-renders.
- Learned to build compound memoized selectors from simpler base selectors.
- Learned to identify and eliminate inline derived-state computation inside components.